Symbolic Time Derivatives in Underworld3

Louis Moresi[1]

[1] Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193596

Abstract

In Underworld3, the time derivative is a symbolic object. It appears in the solver's strong form as a SymPy expression, alongside the constitutive stress and the body force.

Keywords Underworld Code, Tricks of the Trade, Documentation, development

PUBLISHED Apr 16, 2026 ARTICLE UWTN 2026-007 VERSION 1.1.0 LICENCE CC-BY-4.0 ARCHIVED 2026-08-10T04:19:43Z SOURCE https://www.underworldcode.org/symbolic-time-derivatives-in-underworld3/

A symbolic time derivatives: you can inspect it, display it in a notebook, and verify that the time discretisation is doing what you expect. And you can swap between Lagrangian, Semi-Lagrangian, and Eulerian approaches without rewriting the solver. Sympy introduces incredible flexibility for on-the-fly composition of time-dependent problems.

Many geodynamics equations involve a material derivative. Temperature advection-diffusion, viscoelastic stress transport, Navier-Stokes momentum. The material derivative $D\phi/Dt$ combines the time rate of change with advection by the flow:

$$\frac{D\phi}{Dt} = \frac{\partial \phi}{\partial t} + \boldsymbol{v} \cdot \nabla \phi \tag{1}$$

Discretising this equation requires making specific choices. How do you handle the advection term? How many previous timesteps do you use? How do you deal with variable timestep sizes? These choices affect accuracy, stability, and computational cost.

In many finite element codes, these choices are baked into the solver implementation. A classic example is Crank-Nicolson time stepping: the flux must be evaluated at both the current and previous timestep, which means adding history terms to the left-hand side of the assembled system as well as introducing a time-derivative term on the right-hand side. The time discretisation reaches into both sides of the equation, and changing it means restructuring the solver. In UW3, the time derivative ***offers*** flux and force terms to the solver to handle symbolically as it sees fit.

1. The DDt Hierarchy

UW3 provides five implementations of the time derivative, all sharing the same calling interface and working for scalar, vector and tensor quantities.

Eulerian stores history on the mesh. The material derivative is approximated by finite differences in time at fixed grid points, with an advection correction. This is the classical approach: simple, mesh-based, but subject to CFL stability constraints when advection dominates.

Eulerian SUPG also stores history on the mesh, but it does not correct for advection after the fact. It hands the solver an advection term to assemble *inside* the residual, stabilised by SUPG, so the transport is solved implicitly along with everything else. There is no CFL limit. The plain Eulerian flavour above and this one are different answers to the same problem: one corrects the history explicitly and pays a stability condition, the other puts the transport in the equation and does not.

Semi-Lagrangian traces characteristics backward in time. At each mesh node, it asks: where was this material parcel at the previous timestep? It then interpolates the previous solution at that departure point. This is unconditionally stable because the material derivative is evaluated along the characteristic, not at a fixed grid point. There is no CFL constraint, though the user needs to be conscious of accuracy trade-offs inherent in the scheme.

Lagrangian follows particles through the flow. The history is stored on swarm variables and advected with the particles. This is the natural choice when material history matters physically, as in viscoelastic stress transport where the stress tensor must be advected and rotated with the material. Accuracy of this scheme depends upon the quality of the particle-layout (density of particles, presence of gaps).

Symbolic provides pure symbolic history without mesh or swarm storage. It is used internally by the constitutive model system for building expressions that involve time derivatives.

All five produce the same thing: a symbolic expression that can be embedded in a solver's weak form. The solver does not need to know which implementation is providing the time derivative. It sees a SymPy expression for $D\phi/Dt$ and includes those terms when it differentiates the equation system to determine the Jacobians.

2. BDF Schemes: Multi-Step Time Integration

The time discretisation uses backward differentiation formulas (BDF). These are implicit multi-step methods that use solution values from previous timesteps to approximate the time derivative at the current step.

At order 1, this is the backward Euler method:

$$\frac{D\phi}{Dt} \approx \frac{\phi^n - \phi^{n-1}}{\Delta t} \tag{2}$$

At order 2, BDF-2 uses two previous values for second-order accuracy:

$$\frac{D\phi}{Dt} \approx \frac{\frac{3}{2}\phi^n - 2\phi^{n-1} + \frac{1}{2}\phi^{n-2}}{\Delta t} \tag{3}$$

The coefficients $[3/2, -2, 1/2]$ are for constant timesteps. When the timestep varies, the coefficients adapt. If the current timestep is Δt_n and the previous was Δt_{n-1}, with ratio $r = \Delta t_n / \Delta t_{n-1}$, the BDF-2 coefficients become:

$$c_0 = \frac{1+2r}{1+r}, \quad c_1 = -(1+r), \quad c_2 = \frac{r^2}{1+r} \tag{4}$$

This matters in practice. Underworld simulations usually adjust the timestep as the flow evolves.

The coefficients are computed as exact `sympy.Rational` values and wrapped in UWexpression objects. This means they are routed through PETSc's constants array, just like viscosity or density parameters. When the timestep changes, the coefficient values update without recompilation. The compiled C code has the same structure at every step, only the values in the constants array will change.

3. Adams-Moulton Weighting for Fluxes

BDF handles the time derivative of the unknown. But transient problems may also need a time-weighted evaluation of the flux (the spatial operator). The Adams-Moulton (AM) family provides this.

At order 0, the flux is evaluated purely at the current time (fully implicit). At order 1 with $\theta = 1/2$, we have the *Crank-Nicolson* scheme: the flux is averaged between the current and previous timestep. Higher order variants use more history values.

In UW3, the solver's flux time derivative (`DFDt`) provides an `adams _ moulton _ flux()` method that returns the appropriately weighted combination of the current flux and previous flux values as symbolic forms backed by stored evaluations. This expression then appears in the solver's F_1 template as a symbolic expression. Like the BDF coefficients, the AM weights are UWexpressions that update between timesteps.

The composing solver takes these level weights from `DuDt.spatial _ weights()` rather than from a separate `DFDt`, so one manager decides both how the field is transported and how the flux is weighted in time.

The combination of BDF for the time derivative and AM for the flux evaluation gives a family of time integration schemes. BDF-1 with AM-0 is backward Euler. BDF-2 with AM-1 gives second-order accuracy in both the time derivative and the flux evaluation. The user controls this through the `order` parameter when creating the solver.

4. Order Ramping at Startup

A BDF-2 scheme needs two previous solutions. At the start of a simulation, there is only one (the initial condition). UW3 handles this through automatic order ramping.

The DDt object tracks an `effective _ order` that starts at 1 and increases with each completed solve, up to the requested order. On the first timestep, BDF-1 is used regardless of what order was requested. On the second timestep, BDF-2 becomes available. On the third, BDF-3. The transition is automatic and the coefficients adjust accordingly.

History slots are initialised with the current solution value on the first call. This means the first BDF-1 step sees $\phi^{n-1} = \phi^n$, giving a zero time derivative. This is correct for a system starting from rest or from a steady-state initial condition. For impulsive starts, the first-order accuracy of the initial step is usually acceptable because the solution is changing rapidly and the timestep is typically small.

5. What the Solver Sees

Consider advection-diffusion of temperature:

$$\frac{DT}{Dt} = \nabla \cdot (k \nabla T) + H \tag{5}$$

The solver setup looks like this:

```
adv_diff = uw.systems.AdvDiffusion(
    mesh, u_Field=T, V_fn=v,
    order=2,                    # BDF-2 / AM-1
)
adv_diff.constitutive_model.Parameters.diffusivity = k
adv_diff.f = H
adv_diff.solve(timestep=dt)
```

The `order` parameter controls the time discretisation. The solver builds its weak form from two template expressions, F_0 (force-like, paired with the test function) and F_1 (flux-like, paired with the test function gradient). It asks the history manager for each piece:

```
F0 = DuDt.time_derivative() + DuDt.advection() - H
F1 = sum_k w_k * k * grad(T_k) + DuDt.stabilisation_flux(R)
```

The manager decides how transport is done, and the solver does not know. A semi-Lagrangian manager has already moved the field along its characteristics, so it answers zero for `advection()` and for `stabilisation_flux()`, and the pair reduces to the older form:

```
F0 = DuDt.bdf() / delta_t - H
F1 = DFDt.adams_moulton_flux()
```

which is exactly what `uw.systems.AdvDiffusionSLCN` assembles. The Eulerian SUPG manager instead answers with an advection term and a stabilisation flux, and the same solver becomes an implicit Eulerian one.

At **order 1** (backward Euler / fully implicit), these expand to:

$$F_0 = \frac{T^n - T^{n-1}}{\Delta t} - H \tag{6}$$

$$F_1 = k \nabla T^n \tag{7}$$

The flux is evaluated entirely at the current timestep. The system is first-order accurate in time.

At **order 2** (BDF-2 / Crank-Nicolson), the expressions become:

$$F_0 = \frac{\frac{3}{2}T^n - 2T^{n-1} + \frac{1}{2}T^{n-2}}{\Delta t} - H \tag{8}$$

$$F_1 = \frac{1}{2} k \nabla T^n + \frac{1}{2} k \nabla T^{n-1} \tag{9}$$

The flux is now averaged between the current and previous timesteps. Both the time derivative and the flux evaluation are second-order accurate. The history terms T^{n-1} and T^{n-2} are mesh variable symbols managed by the DDt objects. The coefficients ($3/2$, -2, $1/2$, etc.) are UWexpression symbolic constants whose values update each step, if Δt changes, and no recompilation is required.

Note: terms that contain T^n automatically populate the implicit parts of the solver, while history terms are automatically treated explicitly. They are also automatically combined with history terms originating, for example, from the constitutive law.

The solver differentiates through all of this for the Jacobian. The JIT compiler handles it like any other symbolic expression. From PETSc's perspective, the time-stepping machinery is invisible. It sees compiled C functions at quadrature points, with coefficient values arriving through the constants array.

The solve sequence for each timestep is:

1. **Pre-solve**: Update BDF/AM coefficients for the current timestep. For Semi-Lagrangian, trace characteristics to find departure points and sample history values.
2. **Solve**: PETSc SNES solves the assembled system. The compiled C callbacks evaluate the symbolic expressions at quadrature points.
3. **Post-solve**: Shift the history chain. What was ϕ^{n-1} becomes ϕ^{n-2}. The current solution becomes the new ϕ^{n-1}. Increment the solve counter for order ramping.
4. **Solve**: PETSc SNES solves the assembled system. The compiled C callbacks evaluate the symbolic expressions at quadrature points.
5. **Post-solve**: Shift the history chain. What was ϕ^{n-1} becomes ϕ^{n-2}. The current solution becomes the new ϕ^{n-1}. Increment the solve counter for order ramping.

6. Choosing a Time Derivative

The choice of DDt type is a one-parameter decision at solver construction:

```
# Default for advection-diffusion: Eulerian SUPG (implicit, no CFL limit)
adv_diff = uw.systems.AdvDiffusion(mesh, u_Field=T, V_fn=v)

# The semi-Lagrangian solver keeps its own name
adv_diff = uw.systems.AdvDiffusionSLCN(mesh, u_Field=T, V_fn=v)

# Override with Lagrangian (particle-based, requires a swarm)
DTdt = uw.systems.Lagrangian_Swarm_DDt(
    swarm, psi_fn=T.sym,
    vtype=uw.VarType.SCALAR, degree=T.degree,
    continuous=True, order=2
)
adv_diff = uw.systems.AdvDiffusion(mesh, u_Field=T, V_fn=v,
                                   DuDt=DTdt, order=2)

# Or Eulerian (mesh-based, explicit advection correction)
DTdt = uw.systems.Eulerian_DDt(
    mesh, T, vtype=uw.VarType.SCALAR,
    degree=T.degree, continuous=True, order=2
)
adv_diff = uw.systems.AdvDiffusion(mesh, u_Field=T, V_fn=v,
                                   DuDt=DTdt, order=2)
```

A supplied manager fixes the order, so the solver has to be given the same one; a mismatch raises rather than quietly using two different schemes.

The solver does not need to be made aware of which DDt type you chose. It asks the manager for a time derivative, an advection term and a stabilisation flux, and gets SymPy expressions. The physics of the time discretisation is encapsulated in the DDt object. The numerics of the spatial discretisation are encapsulated in the solver. They communicate through symbolic expressions.

Each solver type has a sensible default. Advection-diffusion and Navier-Stokes default to the Eulerian SUPG manager; the semi-Lagrangian solvers keep their `SLCN` names and their place at large Courant numbers, where tracing a characteristic beats stabilising a residual. What decided it was cost and stability rather than accuracy: assembling the transport is several times cheaper per step than tracing characteristics and interpolating, and it does not care what the Courant number is. On smooth translation the semi-Lagrangian scheme is still the more accurate of the two, which is why it keeps its place; the convection benchmarks, where the flow turns and the error accumulates over a full circuit, are where the two draw level on accuracy and the cost difference decides. There is more on that comparison in Two Ways to Move a Field. Stokes' viscoelastic stress history is still semi-Lagrangian, and pure diffusion is still Eulerian. You only need to override the default when your problem requires it.

7. Why This Matters

In UW2, if you wanted to change from explicit particle advection to a semi-Lagrangian scheme, you would need to plug in a different solver. The time discretisation was part of the solver's implementation, not a separable component. When history terms appeared through the constitutive model, we had to develop a matrix of all potential interactions and code them independently.

In UW3, the time derivative is an object you can create, configure, inspect, and swap. The BDF coefficients are visible as symbolic expressions. The history terms are mesh variables you can plot. The AM flux weighting is a symbolic combination you can display in a notebook.

This is the same design principle we described in the constitutive models post: separate the physics from the numerics, connect them through symbolic expressions, and make both sides inspectable. For constitutive models, the boundary is the stress tensor. For time derivatives, it is the BDF/AM expression. In both cases, the solver sees a SymPy expression and does not need to know how it was constructed.

8. History

- **1.1.0** — 2026-09-08 Updated for the composing solvers. `uw.systems.AdvDiffusion` and `NavierStokes` now take their transport from the history manager and default to `EulerianSUPG`; the semi-Lagrangian classes keep the `SLCN` names. The DDt hierarchy gains a fifth flavour and the solver's template expressions are written in the composing form. A runnable example was added (`examples/timestepping.py`), and the reason the default moved is stated as cost and stability rather than accuracy, which is what the measurements show. The scheme theory — BDF, Adams-Moulton, order ramping — is unchanged, and so is the viscoelastic stress history, which is still semi-Lagrangian.
- **1.0.0** — 2026-04-16 · Moresi (2026) First published.

The Underworld project is supported by AuScope and the Australian Government through the National Collaborative Research Infrastructure Strategy (NCRIS). Source code: github.com/underworldcode/underworld3

References

Moresi, L. (2026,). *Symbolic Time Derivatives in Underworld3*. figshare. https://doi.org/10.6084/M9.FIGSHARE.33193596.V1